

Session 06

Apache Hive

SQL, métadonnées et Data Warehouse sur le Data Lake
Hadoop

HIVEQL

METASTORE

PARTITIONS

**HDFS stocke · Spark transforme · Hive
structure et expose en SQL**

**SELECT ... FROM ...**

sur les fichiers du Data Lake

FONCTIONNALITÉS

Where Are We?

Quatre responsabilités distinctes dans la plateforme

HDFS

STORAGE

Fichiers

YARN

RESOURCES

CPU · RAM

Spark

PROCESSING

Pipelines

Hive

SQL / ANALYTICS

Tables · SQL

Hive ne remplace ni HDFS, ni YARN, ni Spark : il ajoute une couche SQL et métadonnées.

FONCTIONNALITÉS

We Already Have the Data

Après Spark, les données existent déjà dans le Data Lake

RAW

Logs bruts
CSV / JSON
forte granularité

PROCESSED

Données enrichies
ORC / Parquet
agrégations Spark

AUDIT

Contrôles
rejets
indicateurs

`/user/<id>/datalake/...`

Le problème n'est plus "où sont les fichiers ?" mais "comment les rendre interrogeables simplement ?".

FONCTIONNALITÉS

What If We Could Query It With SQL?

Passer des chemins HDFS à une vue analytique

```
SELECT
  app_id,
  error_rate
FROM processed_daily_app_metrics
WHERE event_date = '2026-01-15'
ORDER BY error_rate DESC;
```



HIVE



HDFS

SQL décrit le résultat voulu. Hive relie cette requête aux fichiers et aux métadonnées.

Le langage devient déclaratif : l'utilisateur dit **WHAT**, le moteur décide **HOW**.

FONCTIONNALITÉS

Hive in One Sentence



Hive donne une structure SQL aux fichiers stockés dans un stockage distribué.

FONCTIONNALITÉS

What Can Hive Do?

Les fonctionnalités essentielles

SQL Analytics

SELECT · JOIN · GROUP BY

Metadata

bases · tables · schémas

External Tables

exposer des fichiers existants

Partitioning

réduire les données parcourues

Columnar Formats

ORC · Parquet

Data Warehouse

couche analytique organisée

Hive combine SQL, catalogue de métadonnées et organisation analytique du Data Lake.

HiveQL

Un langage SQL pour les données distribuées

```
SELECT app_id, COUNT(*) AS errors
FROM raw_application_logs
WHERE level = 'ERROR'
GROUP BY app_id
ORDER BY errors DESC;
```

SELECT

colonnes

WHERE

filtrer

GROUP BY

agréger

ORDER BY

trier

HiveQL permet d'exprimer des analyses sans écrire un programme MapReduce ou PySpark pour chaque requête.

From Programming to Declarative SQL

WHAT vs HOW

PROGRAMMING

Lire les fichiers
parcourir les données
distribuer le calcul
agréger les résultats

HOW

HIVEQL

```
SELECT app_id, COUNT(*)  
FROM logs  
GROUP BY app_id;
```

WHAT

L'utilisateur décrit le résultat ; Hive compile et planifie l'exécution.

FONCTIONNALITÉS

Hive Is Designed for Analytics

Excellent pour les scans et agrégations, pas pour tous les workloads

GOOD FIT

- Agrégations massives
- Reporting historique
- Analyses batch
- Data Warehouse
- Scans de gros volumes

POORER FIT

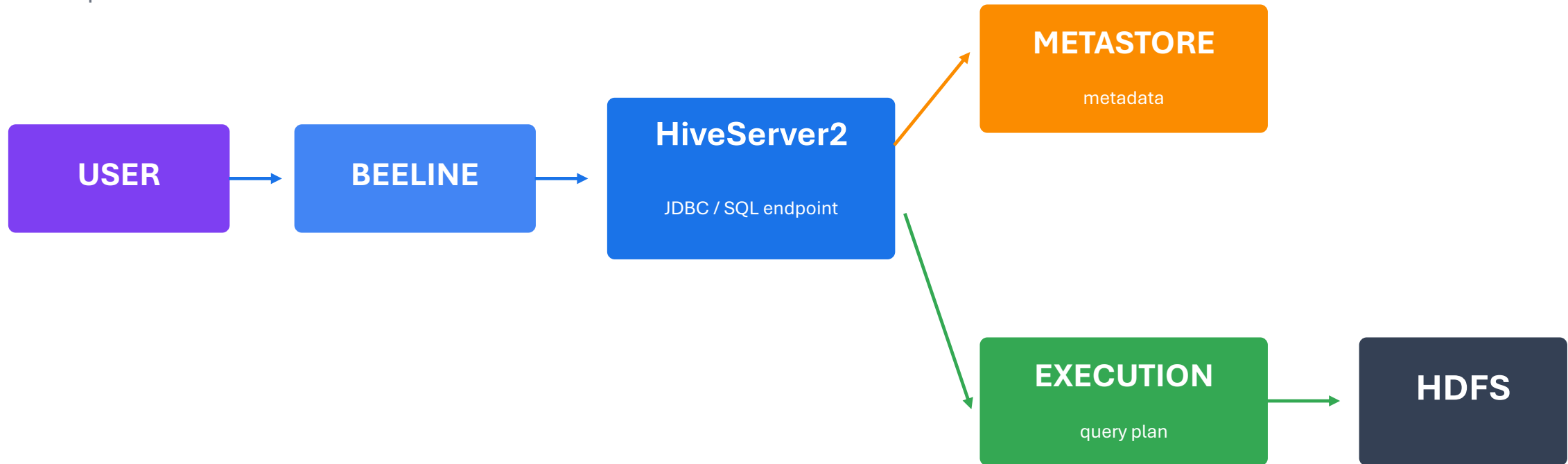
- Recherche ultra-faible latence
- Mises à jour ligne par ligne
- Transactions OLTP fréquentes
- Workloads interactifs très sensibles

Hive est un outil analytique : on optimise le débit et le traitement de grands volumes, pas la latence transactionnelle.

FONCTIONNEMENT

Hive Architecture

Les composants et leurs interactions



Chaque flèche représente une interaction : requête SQL, lookup métadonnées ou lecture/exécution sur les données.

FONCTIONNEMENT

Beeline

Le client SQL du TP06

```
beeline -u
jdbc:hive2://localhost:10000
```



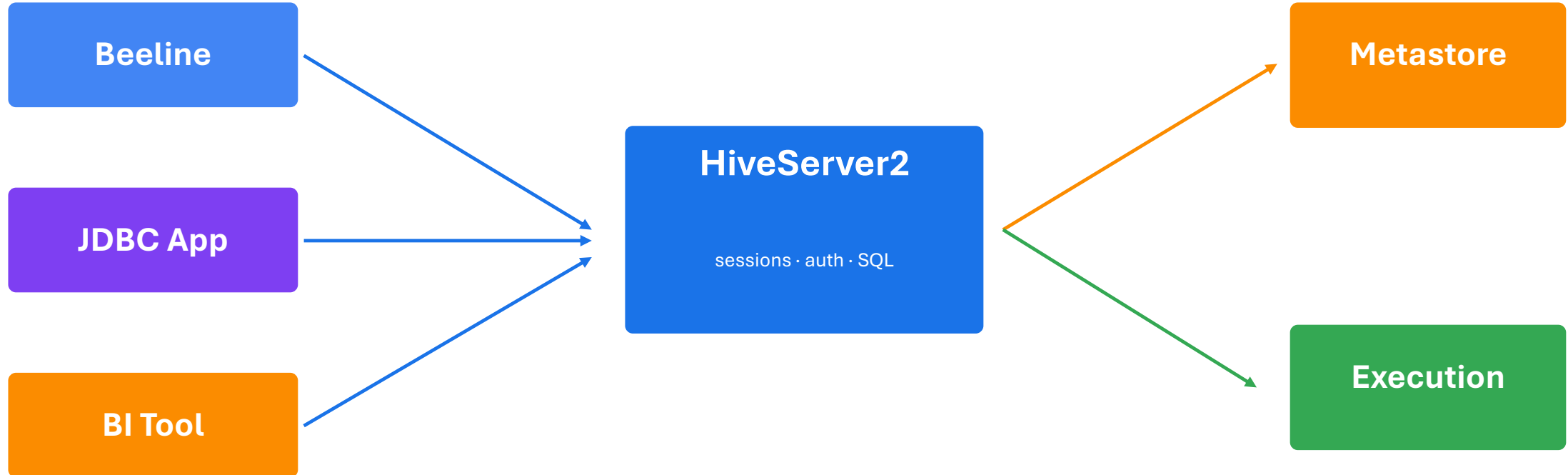
Client	Serveur
psql	PostgreSQL
mysql	MySQL
beeline	HiveServer2

Beeline envoie les requêtes HiveQL à HiveServer2 via JDBC.

FONCTIONNEMENT

HiveServer2

Le point d'entrée serveur pour les clients SQL



HiveServer2 centralise l'accès SQL et dialogue avec le Metastore pour compiler les requêtes.

FONCTIONNEMENT

The Metastore

Le catalogue qui décrit les objets Hive

HIVE
METASTORE

catalogue de métadonnées

Databases

Tables

Columns & Types

Partitions

File Formats

Locations

Table Properties

Le Metastore stocke la description des données - pas nécessairement les données elles-mêmes.

FONCTIONNEMENT

Metadata ≠ Data

La distinction centrale de Hive



Une table Hive peut être une définition de métadonnées qui pointe vers des fichiers existants.

FONCTIONNEMENT

A Hive Table Is a Metadata Definition

Exposer des fichiers existants sans les déplacer

```
CREATE EXTERNAL TABLE logs (  
  app_id STRING,  
  level STRING  
)  
LOCATION '/datalake/raw/logs';
```

METADATA



HDFS FILES

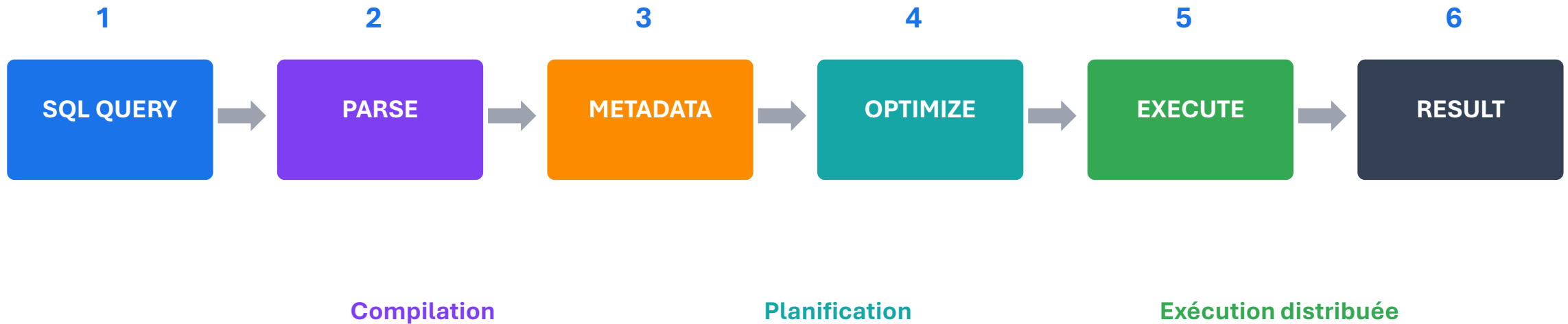
No copy · No migration

CREATE EXTERNAL TABLE crée la définition SQL ; les fichiers peuvent rester là où ils sont.

FONCTIONNEMENT

What Happens When We Execute SELECT?

Le cycle d'une requête Hive



Hive transforme une requête SQL en plan d'exécution sur les données distribuées.

FONCTIONNEMENT

Concrete Query Lifecycle

Exemple : compter les lignes d'une partition

```
SELECT COUNT(*)  
FROM logs  
WHERE year='2026';
```

Parse

Comprendre la requête

Lookup

Table, schéma, partitions

Prune

Écarter les partitions inutiles

Execute

Lire et agréger

Le Metastore permet de localiser les données pertinentes avant que le moteur ne les lise.

FONCTIONNEMENT

Schema-on-Read

Le schéma est appliqué au moment de la lecture



Les fichiers peuvent précéder la table Hive.

Particularité Data Lake : les données peuvent exister avant qu'un schéma Hive ne leur soit appliqué.

PARTICULARITÉS

Managed vs External Tables

Deux modèles de gestion du cycle de vie

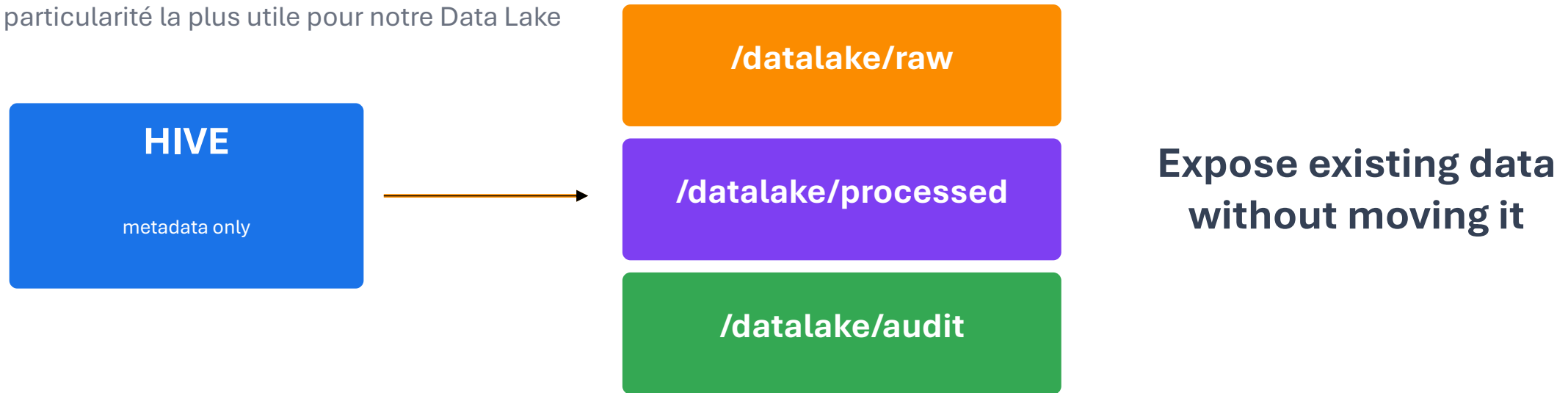
Dimension	Managed / Internal	External
Propriété des données	Hive les gère	Hive les référence
Emplacement	Warehouse Hive	Chemin existant / LOCATION
DROP TABLE	peut supprimer les données	conserve généralement les fichiers
Bon usage	données gérées par Hive	Data Lake / données partagées

Dans un Data Lake partagé avec Spark, les tables externes sont souvent le choix naturel.

PARTICULARITÉS

External Table - The Key Idea

La particularité la plus utile pour notre Data Lake

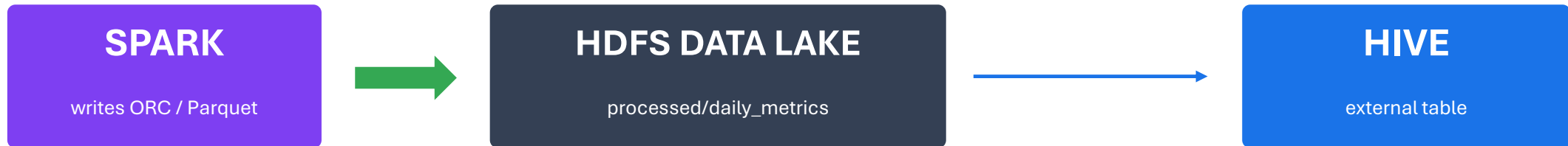


Une table externe sépare le cycle de vie des métadonnées Hive de celui des fichiers.

PARTICULARITÉS

Why External Tables for a Data Lake?

Spark écrit, Hive expose



No copy

No migration

No rewrite

Le même fichier peut être produit par Spark et interrogé ensuite via Hive.

PARTICULARITÉS

The Problem: Years of Logs

Pourquoi organiser les données ?

2024

2025

2026

2027

2028

Query: only 2026-01-15

Faut-il vraiment parcourir toutes les années ?

Le partitionnement permet de réduire les données que Hive doit examiner.

PARTICULARITÉS

Partitioning

Organiser les données selon des valeurs de colonnes

```
logs/  
├── year=2025/  
└── year=2026/  
    ├── month=01/  
    │   ├── day=15/  
    │   └── day=16/  
    └── month=02/
```

Directory layout

Les valeurs de partition apparaissent dans l'arborescence.

Metadata

Le Metastore enregistre les partitions connues.

Query pruning

Un filtre peut éviter des répertoires entiers.

PARTICULARITÉS

Partition Columns May Come From the Path

Les valeurs de partition ne sont pas forcément dans le fichier

```
/application_logs/  
  year=2026/  
  month=01/  
  day=15/  
  logs.csv
```

```
CSV columns:  
  event_ts  
  app_id  
  level  
  status_code  
  ...
```

year / month / day viennent du chemin de partition

La colonne de partition est une métadonnée de placement, pas nécessairement une colonne physique du fichier.

PARTICULARITÉS

Partition Pruning

Read less data

```
WHERE year='2026'  
      AND month='01'  
      AND day='15'
```

2024

2025

2026

01

✗

15

✓

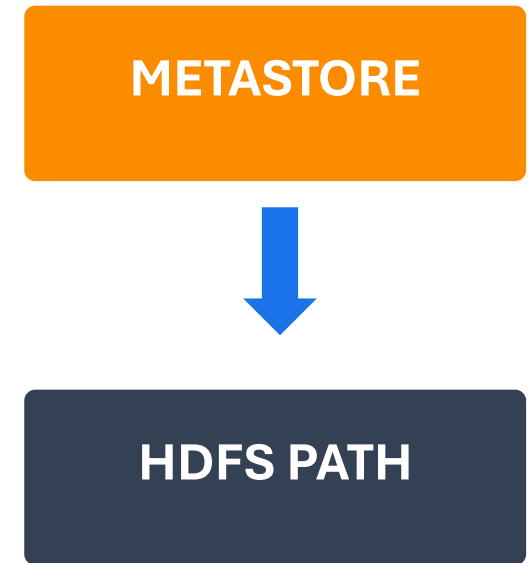
Filtrer sur les colonnes de partition réduit le volume de données lues.

PARTICULARITÉS

Add a Partition

Enregistrer explicitement un nouveau chemin

```
ALTER TABLE raw_application_logs ADD IF NOT EXISTS  
PARTITION (  
  source_team='team_payments',  
  year='2026',  
  month='01',  
  day='15'  
)  
LOCATION '/user/<id>/datalake/raw/.../day=15';
```

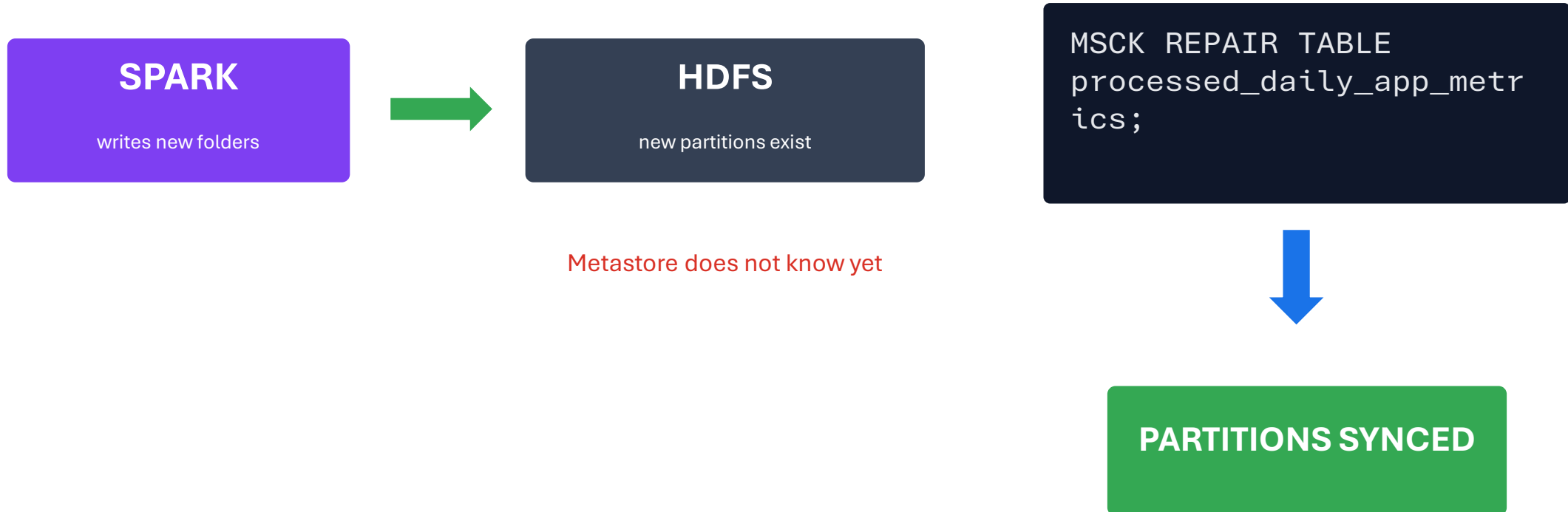


ALTER TABLE ADD PARTITION enregistre une partition précise avec son LOCATION.

PARTICULARITÉS

MSCK REPAIR TABLE

Découvrir des partitions déjà présentes dans l'arborescence



MSCK REPAIR est utile quand les dossiers de partitions existent déjà avant leur enregistrement dans Hive.

PARTICULARITÉS

Partitioning Is Not Bucketing

Deux mécanismes différents

PARTITIONING

Organisation par valeur de colonne
→ souvent des répertoires

Exemple : event_date=2026-01-15

BUCKETING

Distribution par hash dans un nombre fixe de buckets

Exemple : hash(app_id) → 8 buckets

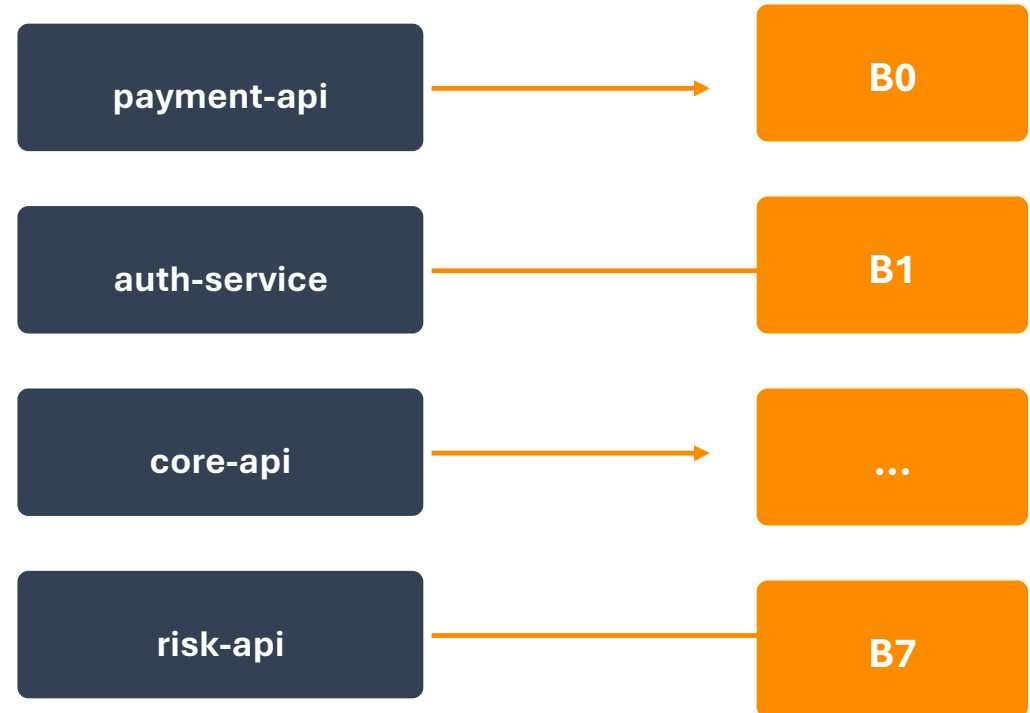
Partition = sélectionner des sous-ensembles ; Bucket = répartir les lignes à l'intérieur d'une table/partition.

PARTICULARITÉS

Bucketing

Hash distribution into a fixed number of buckets

```
CLUSTERED BY (app_id)  
INTO 8 BUCKETS
```



Le bucket est déterminé par une fonction de hash et un nombre fixe de buckets.

PARTICULARITÉS

Partition vs Bucket

Même objectif général d’organisation, mécanismes différents

Dimension	Partition	Bucket
Principe	valeur de colonne	hash d’une colonne
Structure	sous-répertoires	nombre fixe de fichiers/buckets
Exemple	event_date	app_id
But principal	pruning / organisation	distribution / layout

Choisir les partitions d’abord selon les filtres usuels ; utiliser le bucketing pour des besoins plus spécifiques.

PARTICULARITÉS

Text / CSV

Simple à échanger, moins adapté aux analyses massives

STRENGTHS

- Lisible par un humain
- Simple à produire
- Portable
- Bon format d'échange

LIMITS

- Parsing texte
- Schéma faible
- En-têtes / échappements
- Scans plus volumineux

La zone raw peut rester en CSV, mais la zone processed gagne à utiliser un format analytique colonne.

PARTICULARITÉS

Columnar Storage

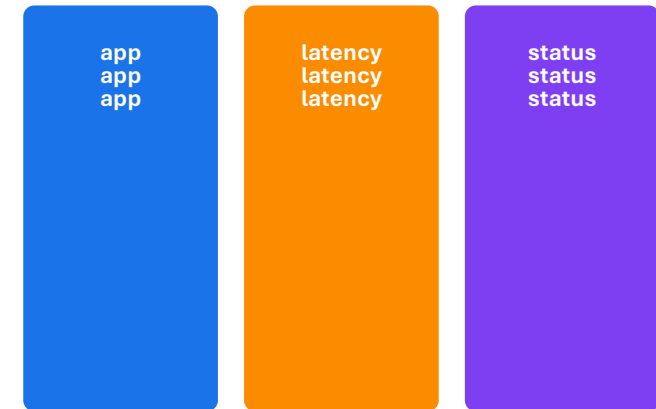
Lire seulement les colonnes nécessaires

ROW-ORIENTED

id	app	latency	status
id	app	latency	status
id	app	latency	status

SELECT app, latency

COLUMNAR



Les formats colonnes réduisent les lectures inutiles pour les requêtes analytiques ciblées.

PARTICULARITÉS

ORC

Optimized Row Columnar - très intégré à l'écosystème Hive

ORC

columnar format

Columnar

lecture sélective

Compression

réduire l'espace

Statistics

aider l'optimiseur

Hive-friendly

format historique de référence

ORC est particulièrement adapté aux tables analytiques Hive et aux scans de colonnes.

PARTICULARITÉS

Parquet

Un autre format colonne largement utilisé

PARQUET

columnar format

Cross-engine

Spark · Hive · Trino...

Compression

réduire I/O

Schema

types structurés

Analytics

column pruning

Parquet est un format colonne très répandu pour les Data Lakes multi-moteurs.

PARTICULARITÉS

CSV vs ORC vs Parquet

Choisir le format selon le rôle de la zone

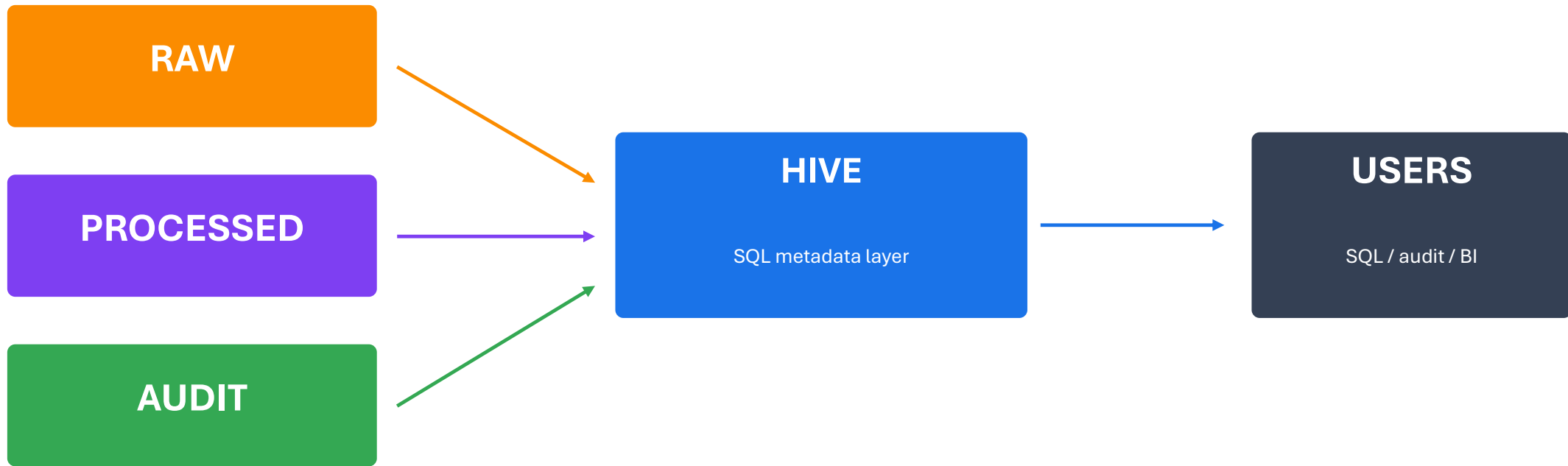
Critère	CSV	ORC	Parquet
Nature	texte	colonne	colonne
Lisibilité humaine	forte	faible	faible
Analytique	basique	très adapté	très adapté
Schéma / types	limité	riche	riche
Usage projet	raw	processed	processed

Le format est un choix d’architecture : raw privilégie la conservation, processed privilégie l’analyse.

PROJET DORA

Back to Our DORA Data Lake

Hive devient la couche SQL au-dessus des zones existantes



Hive n'oblige pas à déplacer le Data Lake : il expose ses fichiers sous forme de tables.

PROJET DORA

Raw Hive Table

Exposer les logs bruts partitionnés

```
raw_application_logs
```

Columns:

```
event_ts  
app_id  
env  
level  
status_code  
response_time_ms  
request_id  
message
```

PARTITIONED BY

source_team

year

month

day

La table raw conserve la granularité originale et utilise les partitions pour cibler les périodes/équipes.

PROJET DORA

Processed Hive Table

Interroger les indicateurs produits par Spark

```
processed_daily_app_metrics
```

```
app_name  
owner_team  
criticality  
business_domain  
event_count  
error_count  
warning_count  
avg_response_time_ms  
max_response_time_ms  
sla_breach_count  
error_rate
```

ORC / PARQUET

PARTITIONS

event_date

app_id

La zone processed est la plus directement adaptée aux analyses Hive : données enrichies, format colonne, partitions utiles.

PROJET DORA

Audit Hive Table

Historiser les contrôles et leurs résultats

Column	Meaning
check_ts	moment du contrôle
check_name	type de contrôle
table_name	table contrôlée
partition_filter	périmètre
row_count	volume observé
status	OK / KO
comment	contexte

partition: check_date

Un contrôle utile à l’audit doit être historisé, contextualisé et reproductible.

Three Zones, Three Purposes

Une organisation lisible du Data Lake

Zone	Purpose	Typical format
RAW	preuve originale / forte granularité	CSV / logs
PROCESSED	données fiables et analytiques	ORC / Parquet
AUDIT	contrôles, rejets, preuves	CSV / ORC / Parquet

La valeur de Hive vient aussi de cette modélisation claire des zones et de leurs responsabilités.

PROJET DORA

From Data Lake to Data Warehouse

Ajouter schéma, tables métier et SQL aux fichiers



Hive construit la couche Data Warehouse du projet sans forcément recopier les fichiers.

Dans ce module, le “Data Warehouse Hive” est surtout une couche SQL/métadonnées organisée sur le Data Lake.

PROJET DORA

Operational Question → SQL

Exemple : quelles applications ont généré des erreurs ?

```
SELECT app_id, COUNT(*) AS error_count
FROM raw_application_logs
WHERE year='2026'
      AND month='01'
      AND day='15'
      AND level='ERROR'
GROUP BY app_id
ORDER BY error_count DESC;
```

OPS VIEW

Granularité brute
Période ciblée
Erreur par application

Une requête d'exploitation répond rapidement à une question opérationnelle ciblée sur les logs bruts.

Audit Question → SQL

Exemple : comparer taux d'erreur et violations SLA

```
SELECT
  app_id,
  error_rate,
  sla_breach_count
FROM processed_daily_app_metrics
WHERE event_date='2026-01-15'
ORDER BY error_rate DESC;
```

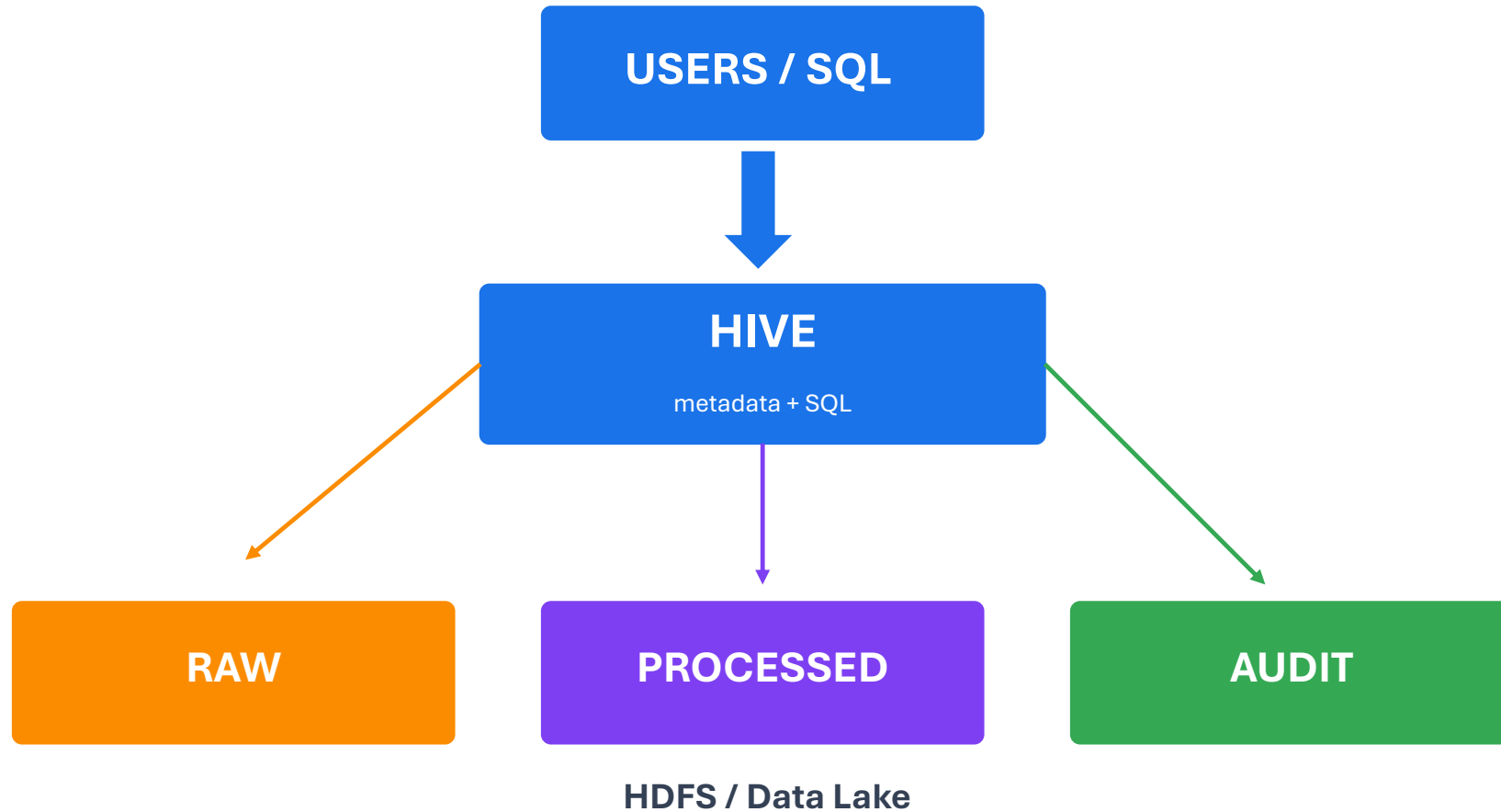
AUDIT VIEW

Indicateurs préparés
Comparaison
Traçabilité
Lecture historique

Une requête d'audit s'appuie davantage sur des indicateurs préparés, historisés et explicables.

Hive Completes Our Data Platform

Une couche SQL au-dessus du stockage et des traitements



HDFS stocke · Spark transforme · Hive structure et expose en SQL.

TP06

TP06 - Hive Database

6 missions pour transformer le Data Lake en espace analytique SQL

1 · CONNECT

Beeline → HiveServer2

2 · MODEL

Database + schemas

3 · EXPOSE

External tables → HDFS

4 · PARTITION

ALTER TABLE · MSCK

5 · QUERY

HiveQL analytics

6 · OPTIMIZE

EXPLAIN · pruning

Livrable : base Hive + tables + partitions + 3 requêtes DORA + justification des choix

Objectif : exposer raw / processed / audit en SQL et comprendre les choix de modélisation Hive.